Synthetic controls — weighted construction of the counterfactual control group

annon for peer review

2025-11-22

1 What this script does

This script turns the methodology in the accompanying note into runnable R. It moves through five stages and each stage has a short explanation, the matching equation, and then the code that implements it.

The five stages are pre-treatment trajectories, within-country standardisation and distance, donor screening (the donor pool), constrained synthetic-control weights, and finally the global donor weights. The last chunk writes a single Excel workbook with two sheets, one holding the donor pool and one holding the weights.

```
library(readxl)
library(dplyr)
library(tidyr)
library(writexl)
library(Rsolnp)
```

2 Data and the pre-treatment window

We read the filtered UCAS data and keep only the pre-Brexit years, meaning every observation from 2020 or earlier. All donor screening and all weight estimation happen inside this window, so nothing post-treatment can leak into the construction of the control group.

The pre-treatment trajectory of a country c is just its sequence of application counts up to 2020.

$$Y_c^{\text{pre}} = \left(\text{App}_{c,2014},\ \text{App}_{c,2015},\ \ldots,\ \text{App}_{c,2020}\right)$$

```
data <- read_excel("data_filtered.xlsx", sheet = "full_data")

# entities that are NOT sovereign foreign countries
non_countries <- c(
  # UK home nations
  "England", "Scotland", "Wales", "Northern Ireland",
  # Crown Dependencies
  "Alderney", "Guernsey", "Jersey", "Isle of Man",
  # British Overseas Territories
  "Anguilla", "Bermuda", "British Antarctic Territory",
  "British Indian Ocean Territory", "British Virgin Islands",
  "Cayman Islands", "Falkland Islands (Malvinas)", "Gibraltar",
  "Montserrat", "St Helena, Ascension & Tristan da Cunha",
  "Turks and Caicos Islands",
  # dependencies / overseas regions of other states
  "Aland Islands", "Aruba", "Canary Islands", "Christmas Island",
  "Curacao", "Faroe Islands", "French Guiana", "French Polynesia",
  "Greenland", "Guadeloupe", "Guam", "Hong Kong", "Macao",
  "Martinique", "Netherlands Antilles", "New Caledonia",
  "Norfolk Island", "Northern Mariana Islands", "Puerto Rico",
  "Reunion", "Sint Maarten (Dutch Part)", "St Barthelemy",
  "St Martin (French part)", "Virgin Islands (US)",
  # non-standard Cyprus codings (Cyprus is already treated via eu_real)
  "Cyprus (Non-European Union)", "Cyprus (Not otherwise specified)",
  # aggregates, placeholders, defunct codes
  "All", "No information provided (Overseas)", "Not Known",
  "Stateless", "USSR (Not In Use)", "French West Indies (Not In Use)"
)

data <- data %>%
  filter(!Domicile_named_country %in% non_countries)
# the 26 treated EU origin markets
eu_real <- c(
  "Austria", "Belgium", "Bulgaria", "Croatia", "Cyprus (European Union)",
  "Czech Republic", "Denmark", "Estonia", "Finland", "France",
  "Germany", "Greece", "Hungary", "Italy",
  "Latvia", "Lithuania", "Luxembourg", "Malta", "Netherlands",
  "Poland", "Portugal", "Romania", "Slovakia", "Slovenia",
  "Spain", "Sweden"
)

# keep only the pre-treatment window
data <- data %>% filter(Year <= 2020 & Year > 2013 )

data <- data %>%
  group_by(Domicile_named_country) %>%
  filter(all(Applicants >= 100)) %>%
  ungroup()
```

3 Building the country-by-year trajectory matrix

We collapse the data to one row per country and one column per year. Each cell is the total number of applicants for that country in that year. This wide matrix is the raw material for every distance and every fit that follows.

```
pre <- data %>%
  select(Domicile_named_country, Year, Applicants)

traj <- pre %>%
  group_by(Domicile_named_country, Year) %>%
  summarise(Applicants = sum(Applicants), .groups = "drop") %>%
  pivot_wider(names_from = Year, values_from = Applicants) %>%
  as.data.frame()

rownames(traj) <- traj$Domicile_named_country
traj <- traj[, -1]            # drop the name column, keep years only

# Logged levels, reused later for donor screening and the synthetic fit.
# Use the ordinary logarithm of the observed applicant count.
logmat <- log(as.matrix(traj))

# log(0) is -Inf and log(negative values) is NaN. Keep only countries
# with finite logged values in every pre-treatment year.
valid_log_rows <- apply(logmat, 1, function(x) all(is.finite(x)))
logmat <- logmat[valid_log_rows, , drop = FALSE]

# A constant pre-treatment trajectory has zero within-country standard
# deviation and cannot be standardised for distance calculations.
row_log_sd <- apply(logmat, 1, sd)
valid_scale_rows <- is.finite(row_log_sd) & row_log_sd > 0
logmat <- logmat[valid_scale_rows, , drop = FALSE]

if (nrow(logmat) == 0L) {
  stop("No countries have complete, positive, non-constant pre-treatment trajectories.")
}

# The standardisation section historically uses the name logtraj.
# Keep it as an explicit alias so both downstream object names exist.
logtraj <- logmat
```

```
#explore the dimensions of the final database

dim(logtraj)
```

```
## [1] 78  7
```

The resulting sample contains 78 countries observed over 7 pre-treatment years

4 Within-country standardisation

Donor screening should depend on the shape of a country's path, not on its size. A large market and a small market that grow in the same pattern should look similar. To get that, we log each series and then standardise it inside each country, subtracting that country's own mean and dividing by its own standard deviation over the pre-period.

$$Z_{c,t} = \frac{\log(\mathrm{App}_{c,t}) - \mu_c}{\sigma_c}$$

Here μ_c and σ_c are computed across the years of country c, not across countries. This is the one place the original script needed a fix. The base `scale()` function standardises columns, which would have standardised each year across countries. We want the opposite, so we transpose, scale, and transpose back, which standardises each country's own row.

```
# t(scale(t(.))) standardises ROW-WISE, i.e. within each country
traj_scaled <- t(scale(t(logtraj)))
```

```
# Convert matrices to data frames and retain row names as a column
#logtraj_df <- as.data.frame(logtraj)
#logtraj_df$country <- rownames(logtraj_df)
#logtraj_df <- logtraj_df[, c("country", setdiff(names(logtraj_df), "country"))]

#traj_scaled_df <- as.data.frame(traj_scaled)
#traj_scaled_df$country <- rownames(traj_scaled_df)
#traj_scaled_df <- traj_scaled_df[, c("country", setdiff(names(traj_scaled_df), "country"))]

# Export to Excel to test for correctness in subsample of countries (DONE)
#write_xlsx(
#  list(
#    logtraj = logtraj_df,
#    traj_scaled = traj_scaled_df
#  ),
#  path = "log_tests.xlsx"
#)
```

5 Pairwise distance and the donor pool

With every country reduced to a standardised shape, we measure how close two shapes are with Euclidean distance over the pre-period.

$$d_{j,k} = \sqrt{\sum_{t \le 2020} \left(Z_{j,t} - Z_{k,t}\right)^2}$$

For each treated EU market j we then keep the five non-EU origins with the smallest distance. That set is the donor pool Γ_j.

$$\Gamma_j = \underset{k \notin \mathrm{EU}}{\arg\min}^{(5)}\, d_{j,k}, \qquad |\Gamma_j| = 5$$

```
# Number of nearest non-EU donors retained for each treated EU market.
n_donors <- 3L

dist_mat <- as.matrix(dist(traj_scaled, method = "euclidean")) #change to euclidean otherwise

treated <- rownames(traj_scaled) %in% eu_real
donors  <- !treated

# rows = treated EU markets, columns = candidate non-EU donors
dist_treated_donors <- dist_mat[treated, donors, drop = FALSE]
eu_present <- rownames(dist_treated_donors)

if (length(eu_present) == 0L) {
  stop("No treated EU countries remain after constructing the logged panel.")
}

if (ncol(dist_treated_donors) < n_donors) {
  stop(sprintf(
    "Only %s eligible donors remain, fewer than n_donors = %s.",
    ncol(dist_treated_donors), n_donors
  ))
}
```

```
# to manually verify everything we run this

#mat_to_df <- function(x, rowname_col = "country") {
#  data.frame(
#    country = rownames(x),
#    as.data.frame(x),
#    row.names = NULL,
#    check.names = FALSE
#  )
#}

#write_xlsx(
 # list(
#    traj_scaled = mat_to_df(traj_scaled),
#    dist_mat = mat_to_df(dist_mat)
# ),
 # path = "matrix_distances.xlsx"
#)
```

6 Constrained synthetic-control weights

Screening tells us which donors are eligible. It does not tell us how to combine them. For that we leave the standardised shapes behind and go back to logged levels, then find the convex combination of the five donors that best tracks the EU market over the pre-period.

$$w_j = \arg\min_{w_j} \sum_{t \le 2020} \Big(\log(\text{App}_{j,t}) - \sum_{k \in \Gamma_j} w_{jk} \, \log(\text{App}_{k,t}) \Big)^2$$

The combination is restricted to be a genuine weighted average, so weights are non-negative and add up to one. Non-negativity stops the fit from extrapolating beyond the donors we actually observe, and the adding-up rule keeps the synthetic market inside the range of real donor paths.

$$w_{jk} \ge 0, \qquad \sum_{k \in \Gamma_j} w_{jk} = 1$$

The helper below states the problem in exactly those terms. The objective is the sum of squared residuals, the equality constraint is the sum of weights, and the bounds give non-negativity. `Rsolnp::solnp` reads almost like the equation.

```
solve_weights <- function(y, Z) {
  n <- ncol(Z)
  fit <- solnp(
    pars  = rep(1 / n, n),                    # start at equal weights
    fun   = function(w) sum((y - Z %*% w)^2),    # squared prediction error
    eqfun = function(w) sum(w),                  # adding-up ...
    eqB   = 1,                                   # ... must equal one
    LB    = rep(0, n),                           # non-negativity
    UB    = rep(1, n),
    control = list(trace = 0)
  )
  as.numeric(fit$pars)
}
```

We loop over the EU markets. For each one we take its five donors, restrict to the years where the market and all five donors are observed, solve for the weights, build the fitted path, and record the root mean squared prediction error.

$$\widehat{Y}_{j,t} = \sum_{k \in \Gamma_j} w_{jk} \, \log(\text{App}_{k,t}), \qquad \text{RMSPE}_j = \sqrt{\frac{1}{T_0} \sum_{t \le 2020} \big(\log(\text{App}_{j,t}) - \widehat{Y}_{j,t} \big)^2}$$

```
#First block. Select the nearest donors only.

donor_selection <- data.frame()

for (j in eu_present) {

  d_row <- dist_treated_donors[j, ]
  d_sorted <- sort(d_row)
  donor_names <- names(d_sorted)[1:n_donors]

  for (k in 1:n_donors) {

    new_row <- data.frame(
      EU_country = j,
      donor_rank = k,
      donor = donor_names[k],
      distance = d_sorted[k],
      stringsAsFactors = FALSE
    )

    donor_selection <- rbind(donor_selection, new_row)
  }
}

rownames(donor_selection) <- NULL

head(donor_selection, n_donors)
```

```
##   EU_country donor_rank              donor distance
## 1    Austria          1 Trinidad and Tobago 1.520189
## 2    Austria          2         New Zealand 1.967550
## 3    Austria          3            Malaysia 2.591757
```

```
#Second block. Find usable years for each EU country and its donors.

year_selection <- data.frame()

for (j in eu_present) {

  donor_names <- donor_selection$donor[donor_selection$EU_country == j]
  selected_countries <- c(j, donor_names)

  block <- logmat[selected_countries, ]

  for (yr in colnames(block)) {

    values_this_year <- block[, yr]
    missing_values <- is.na(values_this_year)
    number_missing <- sum(missing_values)

    if (number_missing == 0) {

      new_row <- data.frame(
        EU_country = j,
        year = yr,
        usable = 1,
        stringsAsFactors = FALSE
      )

      year_selection <- rbind(year_selection, new_row)
    }
  }
}

rownames(year_selection) <- NULL

head(year_selection)
```

```
##   EU_country year usable
## 1    Austria 2014      1
## 2    Austria 2015      1
## 3    Austria 2016      1
## 4    Austria 2017      1
## 5    Austria 2018      1
## 6    Austria 2019      1
```

```
#Third block. Estimate synthetic-control weights.

weight_results <- data.frame()

for (j in eu_present) {

  donor_names <- donor_selection$donor[donor_selection$EU_country == j]
  good_years <- year_selection$year[year_selection$EU_country == j]

  y <- logmat[j, good_years]
  y <- as.numeric(y)

  Z <- logmat[donor_names, good_years]
  Z <- t(Z)

  w <- solve_weights(y, Z)

  for (k in 1:n_donors) {

    new_row <- data.frame(
      EU_country = j,
      donor = donor_names[k],
      synth_weight = w[k],
      stringsAsFactors = FALSE
    )

    weight_results <- rbind(weight_results, new_row)
  }
}

rownames(weight_results) <- NULL

head(weight_results, n_donors)
```

```
##   EU_country              donor synth_weight
## 1    Austria Trinidad and Tobago 2.697885e-09
## 2    Austria         New Zealand 6.484798e-01
## 3    Austria            Malaysia 3.515202e-01
```

```
fit_results <- data.frame()

for (j in eu_present) {

  donor_names <- donor_selection$donor[donor_selection$EU_country == j]
  good_years <- year_selection$year[year_selection$EU_country == j]

  y <- logmat[j, good_years]
  y <- as.numeric(y)

  Z <- logmat[donor_names, good_years]
  Z <- t(Z)

  w <- weight_results$synth_weight[weight_results$EU_country == j]

  fit <- Z %*% w
  fit <- as.numeric(fit)

  errors <- y - fit
  squared_errors <- errors^2
  mean_squared_error <- mean(squared_errors)
  rmspe <- sqrt(mean_squared_error)

  for (i in seq_along(good_years)) {

    new_row <- data.frame(
      EU_country = j,
      year = good_years[i],
      observed = y[i],
      synthetic = fit[i],
      error = errors[i],
      squared_error = squared_errors[i],
      rmspe = rmspe,
      stringsAsFactors = FALSE
    )

    fit_results <- rbind(fit_results, new_row)
  }
}

rownames(fit_results) <- NULL

head(fit_results)
```

```
##   EU_country year observed synthetic        error squared_error      rmspe
## 1    Austria 2014 6.388561  6.433009 -0.044447799  0.0019756068 0.03000786
## 2    Austria 2015 6.445720  6.467246 -0.021526471  0.0004633890 0.03000786
## 3    Austria 2016 6.476972  6.491581 -0.014608225  0.0002134002 0.03000786
## 4    Austria 2017 6.380123  6.386210 -0.006087307  0.0000370553 0.03000786
## 5    Austria 2018 6.437752  6.418909  0.018842306  0.0003550325 0.03000786
## 6    Austria 2019 6.437752  6.401692  0.036059673  0.0013003000 0.03000786
```

```
#Final block. Merge the donor information, distances, weights, and RMSPE.

donor_pool <- merge(
  donor_selection,
  weight_results,
  by = c("EU_country", "donor"),
  all.x = TRUE
)

rmspe_results <- unique(fit_results[, c("EU_country", "rmspe")])

donor_pool <- merge(
  donor_pool,
  rmspe_results,
  by = "EU_country",
  all.x = TRUE
)

donor_pool <- donor_pool[order(donor_pool$EU_country, donor_pool$donor_rank), ]

rownames(donor_pool) <- NULL

head(donor_pool, n_donors)
```

```
##   EU_country              donor donor_rank distance synth_weight      rmspe
## 1    Austria Trinidad and Tobago          1 1.520189 2.697885e-09 0.03000786
## 2    Austria         New Zealand          2 1.967550 6.484798e-01 0.03000786
## 3    Austria            Malaysia          3 2.591757 3.515202e-01 0.03000786
```

```
# we create this for the batch 3 checks in the folder (we replicate the logic with EXCEL SOLVER)

#logmat_df <- data.frame(
#  country = rownames(logmat),
#  as.data.frame(logmat),
#  row.names = NULL,
#  check.names = FALSE
#)

#write_xlsx(
#  list(
#    logmat = logmat_df,
#    donor_selection = donor_selection,
#    weight_results = weight_results
#  ),
#  path = "solver_check_database.xlsx"
#)
```

7 Aggregating to one global weight per donor

The difference-in-differences model uses a single common control path for every EU market, so the country-specific weights have to be collapsed into one weight per donor. We average each donor's weight across the treated markets, treating a donor that never enters a pool as a zero for that market.

$$w_k = \frac{1}{J} \sum_{j=1}^{J} w_{jk}$$

We then normalise so the global weights sum to one. Because each EU market already contributed weights that summed to one, this normalisation is essentially a safety step and barely moves the numbers.

$$\tilde{w}_k = \frac{w_k}{\sum_s w_s}$$

Finally, to give the control side the same total influence as the treated side in the weighted regression, the normalised donor weights are scaled by the number of treated markets J, while every EU observation carries unit weight.

$$\omega_i = \begin{cases} 1 & i \in \Omega \text{ (treated EU)} \\ J \cdot \tilde{w}_k & i \in P \text{ (donor)} \end{cases}$$

```
J <- length(eu_present)   # number of treated markets that got a synthetic control

weights <- donor_pool %>%
  group_by(donor) %>%
  summarise(w_k = sum(synth_weight) / J, .groups = "drop") %>%
  mutate(
    normalized_weight  = w_k / sum(w_k),     # w-tilde
    regression_weight  = J * normalized_weight
  ) %>%
  rename(donor_country = donor, global_weight = w_k) %>%
  arrange(desc(normalized_weight))

weights
```

```
## # A tibble: 26 × 4
##    donor_country            global_weight normalized_weight regression_weight
##    <chr>                           <dbl>             <dbl>             <dbl>
##  1 Korea, Republic of              0.161             0.161              4.18
##  2 Trinidad and Tobago             0.119             0.119              3.08
##  3 Russian Federation              0.108             0.108              2.80
##  4 Malaysia                        0.0973            0.0973             2.53
##  5 Mauritius                       0.0795            0.0795             2.07
##  6 Egypt                           0.0710            0.0710             1.85
##  7 New Zealand                     0.0426            0.0426             1.11
##  8 United States of America        0.0424            0.0424             1.10
##  9 Qatar                           0.0385            0.0385             1.000
## 10 Israel                          0.0385            0.0385             1.000
## # ℹ 16 more rows
```

```
write_xlsx(
  list(
    donor_pool = donor_pool,
    weights    = weights
  ),
  path = "control_group_output.xlsx"
)
```